
e-Portfolio

Künstliche Intelligenz I WS 2025/26

KI-Methoden für den MNIST-Datensatz

Entscheidungsbaum, k-Nächste-Nachbarn, Logistische Regression und
Ensemble-Methoden

Name:	Klossouan Dangui
Gruppennummer:	205
Rolle:	Mitglied (nicht Sprecherin)
Mein Schwerpunkt:	Logistische Regression
Übrige Mitglieder:	Jannik Braun – k-Nächste-Nachbarn Johannes Schmidt – Ensemble Umesh Dahal – Entscheidungsbaum
Sprecherin:	Jannik Braun
Betreuer:	Prof. Dr. Christian Heiliger Dr. Jan-Matthis Waack
Studiengang:	Data Science
Abgabedatum:	15. April 2026

Inhaltsverzeichnis

1	Zusammenfassung des Logbuchs	2
1.1	Fragestellung und Zielsetzung	2
1.2	Ansätze, Methoden und Entscheidungen	2
1.2.1	Datensatz und Vorverarbeitung	2
1.2.2	Experiment 1 – LR mit Datenaugmentierung und GridSearch .	2
1.2.3	Experiment 2 – LR ohne Augmentierung (endgültiges Basismodell)	2
1.2.4	Experiment 3 – PCA-Pipeline und korrigierter GridSearch . . .	3
1.3	Erzielte Ergebnisse	3
1.3.1	Confusion Matrix und Classification Report	4
1.3.2	Lernkurven-Analyse und Underfitting	5
1.3.3	Interpretierbarkeit – Gewichtsvisualisierung	5
1.4	Schwierigkeiten, Grenzen und Ausblick	6
2	Einschätzung der Gruppenmitglieder	7
3	Quellen und Hilfsmittel	8
A	Logbuch – Logistische Regression	9

1. Zusammenfassung des Logbuchs

1.1 Fragestellung und Zielsetzung

Mein individueller Schwerpunkt lag auf der **Logistischen Regression (LR)** zur Klassifikation handgeschriebener Ziffern im MNIST-Datensatz. Die leitende Fragestellung lautete: *Wie gut klassifiziert ein lineares Modell einen nichtlinearen Bildklassifikationsdatensatz, und welchen Einfluss haben Datenaugmentierung, Hyperparameter-Tuning und Dimensionsreduktion auf die Generalisierungsfähigkeit?*

Die konkreten Teilziele umfassten: (1) Vorverarbeitung und Normalisierung des MNIST-Datensatzes, (2) Untersuchung des Effekts von Datenaugmentierung durch Rotation und additives Rauschen, (3) Hyperparameter-Optimierung mittels **GridSearchCV** in zwei Varianten, (4) Dimensionsreduktion mit einer PCA-Pipeline, sowie (5) Analyse der Lernkurve und Interpretation der gelernten Gewichte zur Beurteilung von Bias, Generalisierung und Modellinterpretierbarkeit.

1.2 Ansätze, Methoden und Entscheidungen

Datensatz und Vorverarbeitung

MNIST enthält 60 000 Trainings- und 10 000 Testbilder (28×28 Pixel, Klassen 0–9), geladen via `tensorflow.keras.datasets.mnist`. Pixelwerte wurden durch Division durch 255 auf den Wertebereich $[0, 1]$ normalisiert. Per Flattening wurden die Bilder in 1D-Vektoren der Länge 784 umgewandelt, da die LR keine räumliche Struktur verarbeitet.

Experiment 1 – LR mit Datenaugmentierung und GridSearch

Der Trainingsdatensatz wurde durch Rotation (10° via `cv.warpAffine`) und additives Gaußsches Rauschen ($\sigma = 0,1$) auf 180 000 Bilder verdreifacht. Die Hypothese war, dass mehr und variiere Trainingsdaten die Generalisierung verbessern. Ein **GridSearchCV** (5-fold CV, `n_jobs=-1`) mit $C \in \{0,01; 0,1; 1; 10\}$, `penalty=l2`, `solver=lbfgs` lieferte als bestes Ergebnis $C = 0,01$ mit CV-Accuracy = 85,35 % und Test-Accuracy = 85,10 %.

Während dieses Experiments wurde eine kritische **Inkonsistenz** entdeckt: das Basis-Modell war mit `penalty=None` konfiguriert, der GridSearch-Suchraum beschränkte sich jedoch ausschließlich auf `penalty=l2`. Die gefundenen Hyperparameter galten somit nicht für das eigentliche Modell. Diese Inkonsistenz wurde als Lernschritt dokumentiert und in Experiment 3 systematisch behoben.

Experiment 2 – LR ohne Augmentierung (endgültiges Basismodell)

Nach der überraschend niedrigen Accuracy aus Experiment 1 – trotz dreifachem Datensatz – wurde die Hypothese formuliert, dass Augmentierung durch Rotation der LR schadet. Im Gegensatz zu einem CNN lernt die LR keine geometrische Invarianz; rotierte Bilder erzeugen widersprüchliche Pixelsignale für dieselbe Klasse.

Ohne Augmentierung, direkt auf den originalen 60 000 Bildern mit Standardparametern (`max_iter=500`, `random_state=42`, `l2`, `lbfgs`, $C = 1,0$), stieg die Test-Accuracy auf **92,64 %** – eine Verbesserung von +7,54 Prozentpunkten. Dieses Modell wurde als Basismodell gewählt.

Experiment 3 – PCA-Pipeline und korrigierter GridSearch

Zwei parallele Ansätze wurden verfolgt. Die **PCA+LR-Pipeline** (`sklearn.pipeline.Pipeline`) mit `whiten=True` wurde für $N \in \{50, 100, 150, 200, 300\}$ Komponenten getestet (Tabelle 2). Das Maximum wurde bei $N = 200$ mit 92,64 % erreicht – identisch zum direkten Modell, jedoch bei stark reduzierter Eingabedimension ($784 \rightarrow 200$ Merkmale). Ab $N = 300$ sinkt die Accuracy wieder, da rauschbehaftete Komponenten das Modell negativ beeinflussen.

Der **korrigierte GridSearch** verwendete `solver=saga`, der sowohl L1 als auch L2 unterstützt, mit vollständig kohärentem Parameterraum ($C \in \{0,001; 0,01; 0,1; 1; 10\}$, `penalty \in \{l1, l2\}`). Das beste Modell ($C = 0,1, l2$) erzielte CV-Acc = 92,12 % und Test-Acc = 92,58 % – deutlich über Experiment 1, jedoch leicht unter dem einfachen Basismodell. Besonders auffällig: L1 mit sehr kleinem C (0,001) kollabierte auf nur 69,93 %, da nahezu alle Gewichte auf null regularisiert wurden.

Tabelle 1: Gesamtvergleich aller Experimente. CNN als Referenz aus der Vorlesung.

Modell / Experiment	CV-Accuracy	Test-Accuracy
LR + Augmentierung + GridSearch (lbfgs, l2)	85,35 %	85,10 %
LR ohne Augmentierung (Standardparameter)	–	92,64 %
PCA (200 Komp.) + LR	–	92,64 %
Korr. GridSearch (saga, l2, $C = 0,1$)	92,12 %	92,58 %
CNN (Referenz Vorlesung)	–	$\approx 98\text{--}99\%$

Tabelle 2: PCA+LR-Pipeline – Test-Accuracy nach Anzahl Hauptkomponenten ($C = 1$, lbfgs).

$n_components$	50	100	150	200	300
Test-Accuracy	91,18 %	92,21 %	92,36 %	92,64 %	92,41 %

Tabelle 3: Korrigierter GridSearch – ausgewählte Kombinationen (solver: saga, 5-fold CV).

C	penalty	CV-Accuracy	Std	Rang
0.1	l2	92,12 %	0,41 %	1
1.0	l1	92,04 %	0,45 %	2
1.0	l2	91,94 %	0,48 %	3
0.1	l1	91,49 %	0,46 %	4
0.001	l2	88,91 %	0,61 %	8
0.001	l1	69,93 %	1,03 %	10

1.3 Erzielte Ergebnisse

Confusion Matrix und Classification Report

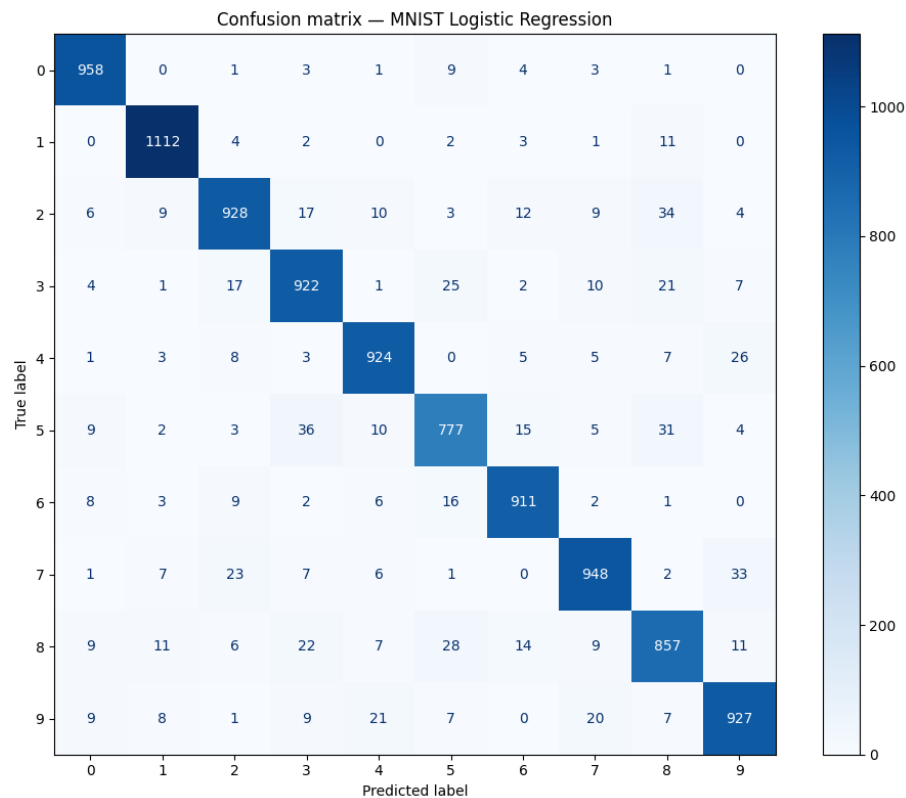


Abbildung 1: Confusion Matrix des Basismodells (LR ohne Augmentierung) auf den 10 000 Testbildern. Insgesamt 736 Fehler (= 7,36 % Fehlerrate, Accuracy 92,64 %). Stärkste Verwechslungspaare: $5 \leftrightarrow 3$, $9 \leftrightarrow 4$, $8 \leftrightarrow 3$. Referenz: [Logbuch, 29. März](#).

Der Classification Report zeigt ausgeglichene Precision und Recall über alle zehn Klassen (macro avg F1 = 0,93). Klassen 5 und 8 erzielten das schlechteste Ergebnis (jeweils F1 = 0,88), was auf ihre ähnliche Kurvenform zurückzuführen ist – ein CNN würde durch seine Faltungsfilter geometrische Unterschiede deutlich besser erfassen. Klasse 1 schneidet mit F1 = 0,97 am besten ab, da die 1 pixelstrukturell kaum Verwechslungspartner hat.

Lernkurven-Analyse und Underfitting

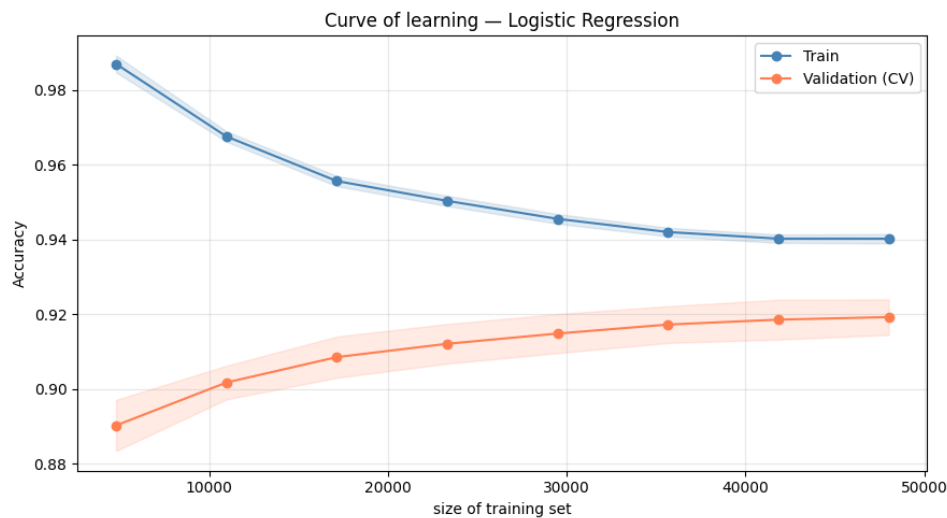


Abbildung 2: Lernkurve der LR: Train-Accuracy (blau) und Validation-Accuracy / 5-fold CV (orange) über 8 Trainingssetgrößen (4 800 bis 48 000 Bilder). Der schattierte Bereich zeigt die Standardabweichung über die Folds. Referenz: [Logbuch, 02. April](#).

Die Lernkurve (Abb. 2) zeigt ein klassisches **Underfitting-Muster (hoher Bias)**: der konstante Abstand zwischen Train- und Validierungskurve signalisiert, dass das Modell die Trainingsdaten selbst nicht vollständig erklären kann. Die Train-Accuracy fällt von $\approx 98,7\%$ bei 4 800 Bildern auf $\approx 94\%$ bei 48 000 Bildern – mehr Daten erhöhen die Aufgabenschwierigkeit für das lineare Modell. Die Validierungskurve flacht bei $\approx 91,9\%$ ab und konvergiert nicht gegen die Trainingskurve. Mehr Daten allein können die strukturelle Schwäche des linearen Modells nicht beheben.

Interpretierbarkeit – Gewichtsvisualisierung

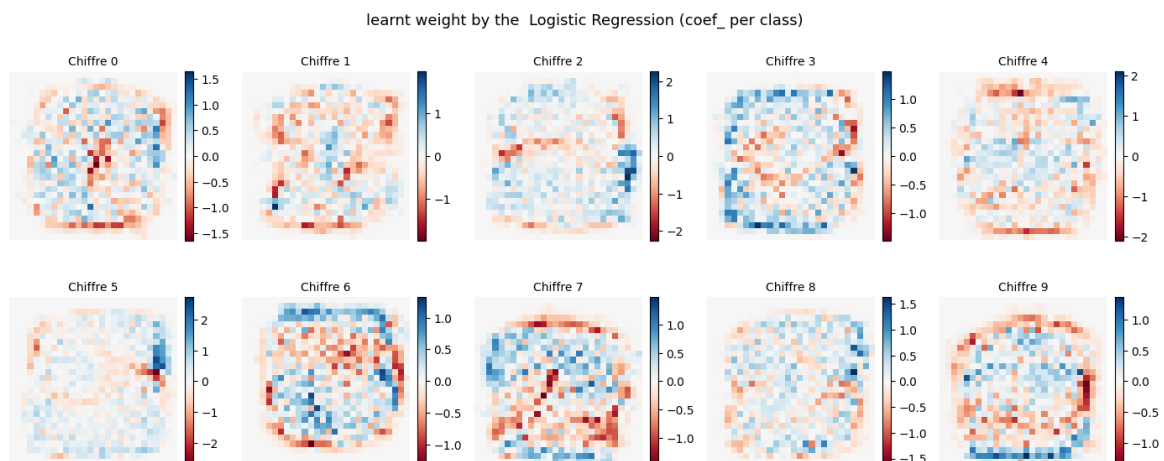


Abbildung 3: Gelernte Gewichte (`coef_` pro Klasse, umgeformt auf 28×28 Pixel). Rot = Pixel, die für die jeweilige Klasse sprechen; Blau = Pixel, die gegen die Klasse sprechen. Die Karten zeigen direkt die Strichstruktur, die das Modell für jede Ziffer erwartet. Referenz: [Logbuch, 03.-04. April](#).

Die Gewichtskarten (Abb. 3) sind ein wesentlicher Vorteil der LR gegenüber neuronalen Netzen: sie sind direkt interpretierbar. Für Ziffer 3 beispielsweise markieren die positiven Pixel (rot) die charakteristischen zwei rechten Haken, während die negativen Pixel (blau) kreisförmige Strukturen repräsentieren, die die Klasse 3 von den Klassen 0, 6 und 8 abgrenzen.

1.4 Schwierigkeiten, Grenzen und Ausblick

Augmentierung schadet linearen Modellen: Rotierte Bilder erzeugen widersprüchliche Pixelmuster für denselben Label. Ein CNN hätte durch seine translationsinvarianten Faltungsfiler von dieser Augmentierung profitiert – die LR nicht, da sie jeden Pixel unabhängig gewichtet.

Strukturelle Accuracy-Decke: Alle Varianten konvergieren bei $\approx 92,6\%$. Die Lernkurve belegt, dass mehr Daten nicht helfen – die LR ist als lineares Modell strukturell nicht in der Lage, die nichtlinearen Entscheidungsgrenzen im Pixelraum abzubilden. CNN-Modelle erreichen auf MNIST über 99%.

PCA ohne zusätzlichen Accuracy-Gewinn: Die PCA-Pipeline bot trotz stark reduzierter Dimensionalität keinen Accuracy-Vorteil. Der Nutzen liegt in reduzierter Trainingszeit und erhöhter Robustheit gegenüber Rauschen in den Rohdaten.

Das **endgültige Modell** ist `LogisticRegression(max_iter=500, random_state=42)` ohne Augmentierung mit **92,64%** Test-Accuracy. Als nächster Schritt bietet sich ein `MLPClassifier` mit einer versteckten Schicht an, um die nichtlinearen Zusammenhänge im Pixelraum zu erfassen und die Accuracy-Decke der LR zu überwinden.

2. Einschätzung der Gruppenmitglieder

Jannik Braun (*k-Nächste-Nachbarn, Sprecherin*). Jannik organisierte alle Gruppentreffen zuverlässig und kommunizierte Zwischenergebnisse stets klar und strukturiert, was den gemeinsamen Fortschritt wesentlich unterstützte. Als Schwäche ist anzumerken, dass gelegentlich Entscheidungen ohne vollständige Rücksprache mit allen Mitgliedern getroffen wurden, was zu kleineren Missverständnissen führte.

Johannes Schmidt (*Ensemble-Methoden*). Johannes arbeitete sehr strukturiert und lieferte seine Ergebnisse stets vollständig und fristgerecht; seine Erklärungen zu Ensemble-Methoden während der Treffen waren für die gesamte Gruppe wertvoll. Verbesserungspotenzial besteht darin, eigene Ideen aktiver in die Gruppendiskussion einzubringen, da er sich häufig zurückhielt.

Umesh Dahal (*Entscheidungsbaum*). Umesh zeigte ein solides theoretisches Verständnis der Entscheidungsbaumalgorithmen und lieferte seine Ergebnisse letztendlich in akzeptabler Qualität ab. Allerdings erschien er regelmäßig zu spät zu den Gruppentreffen oder fehlte ohne vorherige Abmeldung, was die Koordination erschwerte und die Diskussion der Zwischenergebnisse verzögerte; zudem wurden Aufgaben überwiegend erst nach expliziter Aufforderung begonnen.

3. Quellen und Hilfsmittel

Datensätze

- LeCun, Y., Cortes, C., & Burges, C. J. C. (1998). *The MNIST database of handwritten digits*. <http://yann.lecun.com/exdb/mnist/> [Zugriff: März/April 2026]

Bibliotheken und Software

- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830. scikit-learn v1.x. <https://scikit-learn.org> [Zugriff: März/April 2026]
- Abadi, M. et al. (2016). TensorFlow v2.x. <https://www.tensorflow.org> [Zugriff: März 2026]
- Hunter, J. D. (2007). Matplotlib. *Comp. Sci. Eng.*, 9(3), 90–95. <https://matplotlib.org> [Zugriff: März/April 2026]
- Harris, C. R. et al. (2020). NumPy. *Nature*, 585, 357–362. <https://numpy.org> [Zugriff: März/April 2026]
- OpenCV v4.x. <https://opencv.org> [Zugriff: März 2026]
- Python 3.11. <https://www.python.org> [Zugriff: März/April 2026]

Vorlesungsunterlagen

- Heiliger, C. & Waack, J.-M. (2025/26). Vorlesungsfolien: Künstliche Intelligenz I. WS 2025/26, Fachbereich Informatik.

Einsatz generativer KI

Im Rahmen dieses Projekts wurde **Claude (Anthropic, claude.ai)** als Hilfsmittel eingesetzt – ausschließlich zur Unterstützung beim Debugging und bei der L^AT_EX-Redaktion, nicht zur Generierung von Ergebnissen.

Beispielhafte Prompts:

- “Mein *GridSearchCV* gibt eine *ConvergenceWarning* aus bei *max_iter=200*. Wie behebe ich das und warum tritt das auf?”
- “Erkläre mir die Inkonsistenz zwischen *penalty=None* im Basismodell und *penalty=l2* im Suchraum meines *GridSearch*.”
- “Hilf mir, diese L^AT_EX-Tabelle mit *booktabs* korrekt zu formatieren: [Tabelleninhalt].”

Alle inhaltlichen Entscheidungen (Modellwahl, Interpretation der Ergebnisse, Schlussfolgerungen) wurden eigenständig erarbeitet. KI-generierte Vorschläge wurden stets geprüft und manuell übernommen.

A. Logbuch – Logistische Regression

Chronologische Dokumentation meines Arbeitsprozesses, direkt basierend auf den Zellen des Jupyter Notebooks. Stichpunktartige Formulierungen sind gemäß Aufgabenstellung ausdrücklich ausreichend. Das Logbuch zählt nicht zum Seitenumfang des e-Portfolios.

16. Februar 2026 – Erstes Gruppentreffen: Kennenlernen und Projektplanung

Erstes Gruppentreffen (16.02.2026) – Kennenlernen, Rollenverteilung und grobe Planung.

- Gegenseitiges Kennenlernen und Besprechung der Projektanforderungen
- Aufgabenteilung: Schwerpunkt Logistische Regression gewählt
- Grobe Zeitplanung vereinbart; Arbeitsbeginn für Mitte März geplant
- Entwicklungsumgebung festgelegt: Jupyter Notebook, Python, scikit-learn, TensorFlow, OpenCV

23. März 2026 – Projektstart: Datenladen, Normalisierung, Augmentierung und Experiment 1

- Alle Imports gesetzt: `pandas`, `numpy`, `matplotlib`, `tensorflow`, `cv2`, `sklearn` (KNeighborsClassifier, `accuracy_score`, `confusion_matrix`, `classification_report`, `Scaler`, `cross_val_score`, `GridSearchCV`, ...)
- Hilfsfunktionen implementiert:
 - `relu(x)`, `linear(x)` – Aktivierungsfunktionen
 - `conv(pic, conv_matrix, af, bias)` – manuelle Faltung
 - `plot_samples(xdata, ydata, amount=40)` – Visualisierung
 - `rotate_image(image, rotation_angle, scale)` – OpenCV-Rotation
 - `add_noise(img, sigma=0.1)` – Gaußsches Rauschen
- MNIST geladen via `tf.keras.datasets.mnist`: 60 000 Train- / 10 000 Testbilder, 28×28 Pixel
- Normalisierung: Division durch 255 $\Rightarrow$ Wertebereich $[0, 1]$
- Augmentierungsstrategie angewendet: Original + Rotation (10°) + Gaußsches Rauschen ($\sigma = 0,1$) $\rightarrow$ 180 000 Bilder
- Flattening: $(N, 28, 28) \rightarrow (N, 784)$ für Train- und Testset
- **Experiment 1:** `GridSearchCV` (5-fold CV, $C \in \{0,01; 0,1; 1; 10\}$, `penalty=l2`, `solver=lbfgs`) auf augmentierten Daten
- **Ergebnis:** bestes $C = 0,01$, CV-Accuracy = 85,35 %, Test-Accuracy = 85,10 %
- **Problem erkannt:** Basis-Modell hat `penalty=None`, `GridSearch` sucht nur `l2` – Inkonsistenz $\rightarrow$ Korrektur geplant
- **Hypothese:** Augmentierung schadet der LR (85 % trotz $3 \times$ Daten)

27.–28. März 2026 – Experiment 2: LR ohne Augmentierung

- Datensatz neu geladen – originale 60 000 Bilder, ohne Augmentierung
- Modell: `LogisticRegression(max_iter=500, random_state=42)` (Standardparameter: `l2`, `lbfgs`, $C = 1,0$)
- **Ergebnis:** **Test-Accuracy** = 92,64 % – Verbesserung +7,54 PP
- **Erkenntnis:** LR kann keine geometrische Invarianz lernen; Rotation erzeugt

widersprüchliche Pixelsignale pro Klasse

- **Entscheidung:** dieses Modell als Basismodell gewählt

29. März 2026 – Confusion Matrix und Fehleranalyse

- `ConfusionMatrixDisplay.from_predictions` auf Testset berechnet
- 736 Fehler von 10 000 Bildern (= 7,36 % Fehlerrate)
- `classification_report`: macro avg F1 = 0,93; schlechteste Klassen: 5 und 8 (je F1 = 0,88)
- 40 falsch klassifizierte Bilder visualisiert
- Häufigste Verwechslungspaare: $5 \leftrightarrow 3$, $9 \leftrightarrow 4$, $8 \leftrightarrow 3$, $4 \leftrightarrow 6$, $2 \leftrightarrow 7$

30. März – 01. April 2026 – PCA-Pipeline und korrigierter GridSearch

- **PCA+LR Pipeline** mit `sklearn.pipeline.Pipeline`: `PCA(n_components=N, whiten=True) + LogisticRegression(C=1, solver='lbfgs', max_iter=1000)`
- $N \in \{50, 100, 150, 200, 300\}$ getestet – Maximum bei $N = 200$ mit 92,64 %, danach sinkend (Overfitting auf Rauschen)
- **Korrigierter GridSearch** mit `solver='saga'` ($L1+L2$), $C \in \{0,001; 0,01; 0,1; 1; 10\}$, `penalty` $\in \{l1, l2\}$
- Ergebnis: $C = 0,1$, $l2$, CV-Acc = 92,12 %, Test-Acc = 92,58 %
- $L1$ bei kleinen C sehr schwach (69,9 %); $l2$ deutlich stabiler

02. April 2026 – Lernkurven-Analyse

- `learning_curve`: 5-fold CV, 8 Punkte (4 800 bis 48 000 Bilder)
- Train-Accuracy: 98,7 % $\rightarrow$ 94 % (mehr Daten = härtere Aufgabe)
- Val-Accuracy: 88,9 % $\rightarrow$ 91,9 %, flacht ab
- Konstanter Gap $\rightarrow$ **Underfitting** / **hoher Bias**
- Kommentar: *“If train $\approx$ val $\rightarrow$ underfitting. If train $\gg$ val $\rightarrow$ overfitting. On MNIST+LR typically bias – a more complex model would help.”*

03.–04. April 2026 – Gewichtsvisualisierung und Interpretierbarkeit

- Alle 10 Gewichtsvektoren `lr.coef_[k].reshape(28,28)` visualisiert (RdBu-Colormap, symmetrische Farbskala)
- Karten zeigen direkt die Strichstruktur jeder Ziffer
- Detailanalyse für Ziffer 3:
 - Positive Pixel (rot): die zwei rechten Haken der 3
 - Negative Pixel (blau): kreisförmige Struktur (Abgrenzung zu 0, 6, 8)
- Kommentar: *“Red = pixels pushing toward this digit. Blue = pixels pushing toward other classes. This is the main advantage of LR: fully interpretable.”*

10. April 2026 – Zweites Gruppentreffen (online, Discord)

Zweites Gruppentreffen (10.04.2026, online via Discord) – Besprechung der individuellen Zwischenergebnisse.

- Präsentation der bisherigen LR-Ergebnisse (92,64 %, Lernkurve, Gewichtskarten) vor der Gruppe
- Vergleich mit Zwischenergebnissen der anderen Methoden (kNN, Ensemble, Entscheidungsbaum)

- Aufgabenverteilung für die letzten Tage vor der Abgabe beschlossen
-

13.–14. April 2026 – Abschluss und drittes Gruppentreffen

Drittes und letztes Gruppentreffen (14.04.2026, online via Discord) – Zusammenführung der Ergebnisse.

- Alle Experimente abgeschlossen und dokumentiert
- **Endgültiges Modell:** `LogisticRegression(max_iter=500, random_state=42)`, Test-Accuracy = 92,64 %
- Zusammenführung aller Gruppenergebnisse mit der Sprecherin
- e-Portfolio und Logbuch finalisiert; Abgabe am 15. April 2026 vorbereitet